

UVM Testwriter Lab

Week 14 Lab Document

Lab Manual

Revision 1.0

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;

- The publication may not be modified in any way;

- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents
UVM Testwriter Lab

Quick Reference: Project Overview5

Before You Begin7

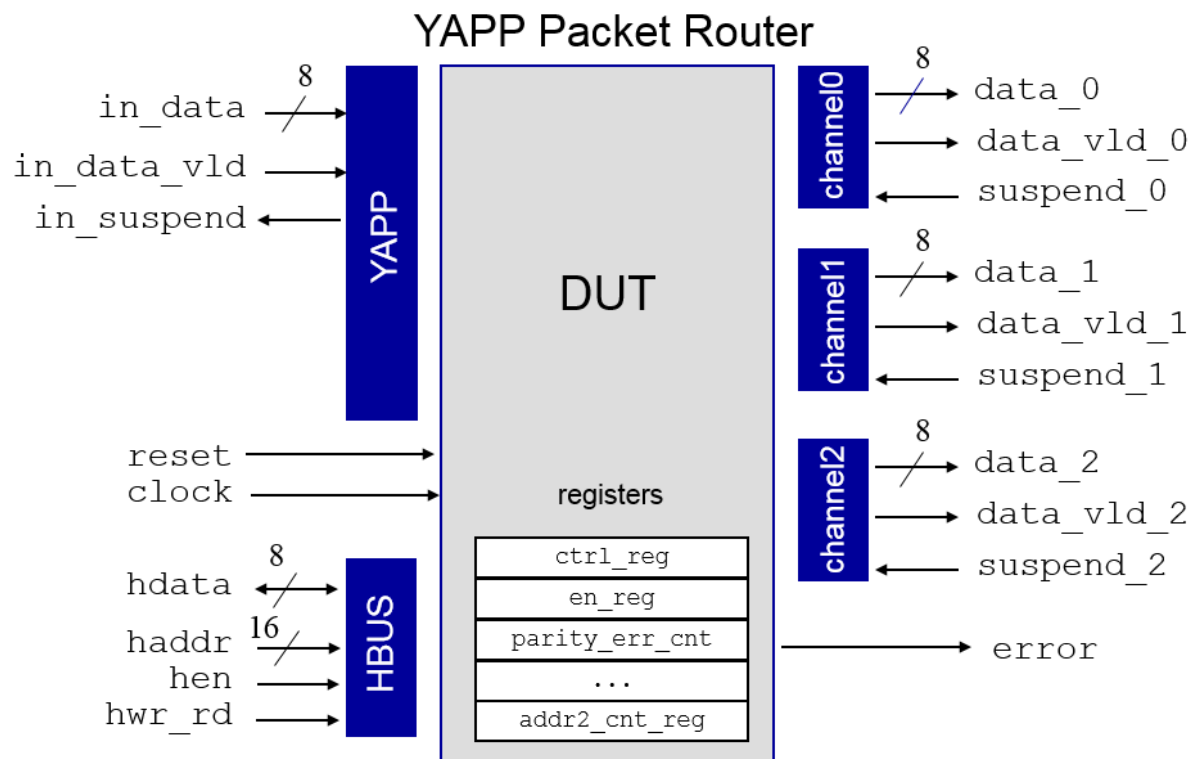
Lab 1 Creating Stimulus with UVM Sequences9

Quick Reference: Project Overview

YAPP Router Description

The YAPP router accepts data packets on a single input port, `in_data`, and routes the packets to one of three output channels: `channel0`, `channel1` or `channel2`. The input and output ports have slightly different signal protocols. The router also has an HBUS host interface for programming registers that are described in the next section.

High-Level Diagram – YAPP Router (Yet Another Packet Protocol)



Packet Data Specification

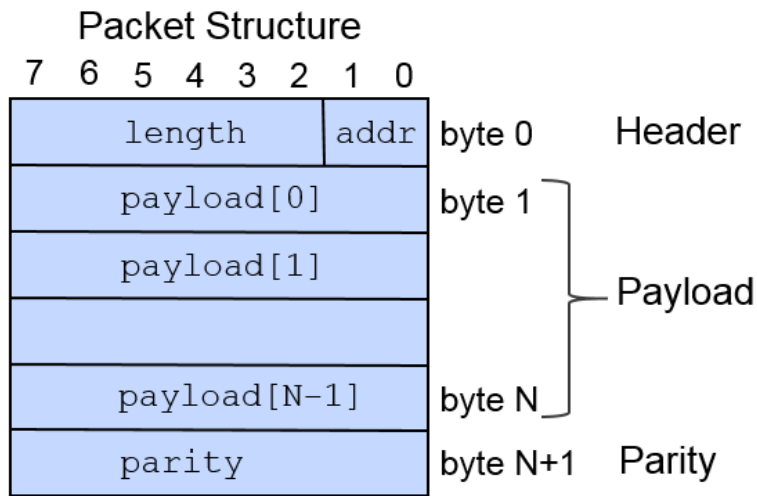
A packet is a sequence of bytes with the first byte containing a header, the next variable set of bytes containing payload, and the last byte containing parity.

The header consists of a 2-bit address field and a 6-bit length field. The address field is used to determine which output channel the packet should be routed to, with the address 3 being illegal. The length field specifies the number of data bytes (payload).

A packet can have a minimum payload size of 1 byte and a maximum size of 63 bytes.

Project Overview

The parity should be a byte of even, bitwise parity, calculated over the header and payload bytes of the packet.



$$1 \leq N \leq 63$$

Before You Begin

Finding Your Files

The UVM training lab database includes the following directories and files:

<code>uvma_training_xx/uvvm</code>	Top-level directory for the training
<code>lab01_setup/</code>	Lab directory where you will be working.
<code>test_install</code>	Simple example to check your UVM installation.

Accessing UVM

UVM 1.2 is supported by XCELIUM 17.04 onwards.

For all simulator releases, there are two options for accessing the UVM library:

- ◆ Use one of the libraries provided with your simulator installation.
- ◆ Download the library from `accelera.org`.

The first option is the best solution for Cadence® users, as the library contains extra Cadence® debug additions for UVM, such as transaction recording.

Setting Up UVM

1. Set an environmental variable to the install path of the required UVM library release, e.g., for the UVM library provided with Cadence simulators:

```
setenv UVMHOME `ncroot`/tools/methodology/UVM/CDNS-1.1d
```

We currently recommend UVM1.1d for *training* due to transaction issues in UVM1.2.

Testing Your UVM Installation

2. Change the directory to `test_install`.
3. Run the test by executing the following command:

```
xrun -f run.f
```
4. In the output log file (`xrun.log`), check the version number of the UVM library. You should see:

```
-----  
UVM-1.x  
...
```

Where `UVM-1.x` is the UVM library version you are using.

5. In the test output, check the installation test passed. You should see:

```
UVM_INFO    ...    UVM TEST INSTALL PASSED!
```

Accessing Help Files

You may wish to bookmark the following file in a web browser to give convenient access to the UVM HTML documentation

```
$UVMHOME/sv/docs/html/index.html
```


Lab 1 Creating Stimulus with UVM Sequences

Objective: To use UVM sequences to define and execute stimulus on a DUT.

For this lab, you will write and execute sequences for an existing UVM environment.

Testing Your UVM Installation

1. Change directory to `lab01_setup/tb`.
2. Check the `run.f` file executes `base_test` (i.e. contains the line `+UVM_TESTNAME=base_test`).
3. Run an Xcelium simulation by executing the following command:

```
xrun -f run.f
```
4. Check the simulator log reports generated by the simulation. The log is also written to the file **`xrun.log`**, which you could open in an editor.

Find the topology print. Use the topology print to find the full hierarchical pathname from (but not including) the topmost test class (`uvm_test_top`) to (and including) the YAPP UVC sequencer and write it below.

Sequencer pathname: _____

Running a Simple Sequence

We supply a simple sequence to get you started and verify the sequence connection.

5. Open the file `lab01_setup/sv/yapp_tx_seqs.sv` and find the sequence `yapp_5_packets`. This sequence uses a simple repeat loop to execute a `uvm_do` macro on the req YAPP packet handle five times.
6. Open the file `lab01_setup/tb/router_test_lib.sv` in a text editor.
7. Find the `connect_phase` function.
 - a. Uncomment the assignment to `yapp_seqr`.

- b. Complete the assignment by setting `yapp_seqr` to the hierarchical pathname of the YAPP sequencer that you read from the topology report and recorded in the step above.

8. Find the `build_phase` function.

- a. Uncomment the `create` call on the `yapp5` handle. This is similar to a new constructor call but creates the sequence instance using the UVM factory, which gives much greater flexibility and control.

```
yapp5 = yapp_5_packets::type_id::create("yapp5", this);
```

Note the syntax of the `create` call for sequence handle names and types:

```
<name> = <type>::type_id::create("<name>", this);
```

9. Find the `run_phase` function.

- a. Uncomment the `start` call on the `yapp5` handle.

Note the syntax of the `start` call.

```
<sequence_handle> = start(yapp_seqr);
```

10. Run a simulation in the `lab01_setup/tb` directory using:

```
xrun -f run.f
```

Your UVC should now generate and print YAPP packets. Check the correct number of packets are printed. Check the address of the packets and the party type.

11. Add the following compilation option to the end of your command line:

```
+SVSEED=random
```

This sets a random value for the initial randomization seed of the simulation. Re-run the simulation, and you should see different packet data. The simulation reports the actual seed used for each simulation in the log file.

Write Your Own Sequences

Now is the time to write your own sequences and execute them on the YAPP UVC sequencer.

We will give you specifications for a number of different, simple stimuli for the YAPP. Write, compile, and start the sequences one at a time. Do as many as you can in the time available.

You will be editing the sequences in the file `lab01_setup/sv/yapp_tx_seqs.sv`, updating `base_test` in the file `lab01_setup/tb/router_test_lib.sv`, and running the simulation in the `tb` directory.

For each of the sequences:

12. Copy the `yapp_5_packets` sequence in `yapp_tx_seqs.sv`, and paste at the end of the file.
13. Update the sequence name. This requires FIVE changes:
 - a. Change the class type name after the `class` keyword.
 - b. Change the default assignment to the string argument `name` in the function `new` constructor.
 - c. Change the reference to the class name in the ``uvm_info` report.
 - d. Change the optional name reference in the `endclass` line. Remember to keep the colon between `endclass` and `name`.
14. Update the `body()` task using the presentation slides and tips below to apply the required stimulus.
15. Update `router_test_lib.sv` to execute the sequence as follows:
 - a. Add a handle for the sequence type, directly under the `yapp5` handle.
 - b. Create an instance for the sequence in the `build_phase` using a `create` call.
 - c. Modify the `start` call in `run_phase` to use your new sequence handle.
16. Test your sequence in simulation in the `lab01_setup/tb` directory using:

```
xrun -f run.f
```

Check the correct packets are printed.

Sequence Specifications

17. Create a single packet sequence to address 1 using a randomization constraint:
yapp_1_seq – single packet to address 1
(`addr==1`)
18. Create a multi-packet sequence with different constraints for each packet:
yapp_012_seq – three packets with incrementing addresses
(`addr==0; addr==1; addr==2`)

19. Create a nested sequence.

yapp_111_seq – three packets to address 1
(do yapp_1_seq three times)

20. Create a sequence to generate a **single** packet with incrementing payload data.

yapp_incr_payload_seq –
Create a single packet to send.
Randomize the packet.
Set the payload values of the single packet to increment from 0 to (length -1).
Update parity.
Send the packet.
Hint: Use ``uvm_create` and ``uvm_send` macros.

21. Create a single sequence to execute all of your new sequences as nested sequences.

yapp_exhaustive_seq – execute all your sequences
(Do all of your user-defined sequences).

Using meaningful handle names for the sequences will help in debug.

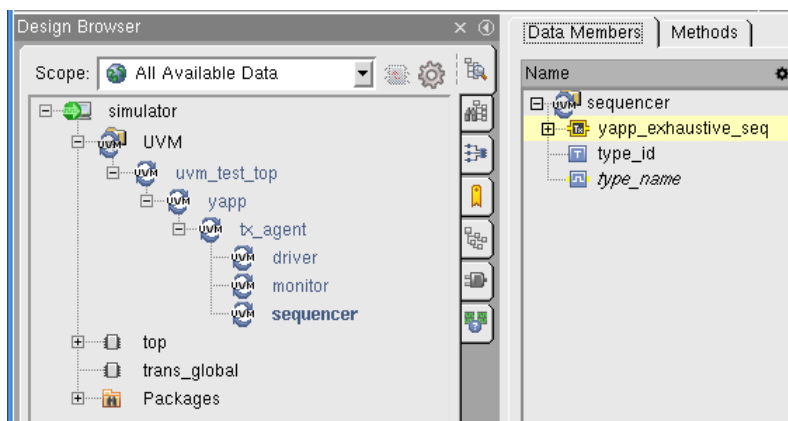
Transaction Debug (SimVision)

22. For your exhaustive sequence, run a SimVision GUI simulation by adding the following xrun command line options

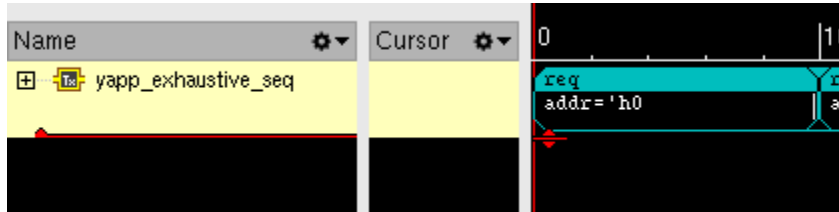
`-gui -access rwc`

23. Run the simulation to the end.

24. In the Design Browser, navigate down the UVM hierarchy to your sequencer instance and find the sequence transaction (same name as your sequence). Add this to the waveform viewer.



25. Click  on the left of the transaction name to expand the sequence hierarchy and pulldown the red-bar  under the `req` transaction to see the packet data.



 End of Lab